PIELAND

APPS/API

Guía del backend de SNAC Hunters y cómo pasar del servidor local a AWS Lambda



Equipo 13 · Taller y Desarrollo de Proyectos Full Stack

Complemento del *Manual de la Demo D3* · Rama `feat/demo-viabilidad-d3`

Contenido

Parte A: Entender `apps/api`

1. Qué es y por qué existe
2. La idea de diseño: un núcleo y dos adaptadores
3. Recorrido completo de una partida validada
4. Archivo por archivo
5. Referencia de endpoints
6. Validación de entrada y códigos HTTP
7. Pruebas
8. Probar la API a mano

Parte B: Del servidor local a AWS Lambda

9. Qué cambia y qué no
10. Cómo CDK convierte el código en una Lambda
11. Requisitos previos
12. Despliegue paso a paso
13. El pendiente del login (importante)
14. Verificar, depurar y errores comunes
15. Costos y cómo borrar todo
16. Resumen para explicarlo

> **Antes de empezar:** los comandos se ejecutan desde la raíz del repositorio (`SNAC-Hunters/`). El prefijo `pnpm --filter @snac-hunters/api` significa "ejecuta esto solo en el paquete `apps/api` ".

Parte A

Entender `apps/api`

1. Qué es y por qué existe

`apps/api` es el **servidor** (backend) del juego. La app web le habla por HTTP. Tiene tres trabajos:

1. **Validar partidas** para que nadie pueda hacer trampa.
2. **Guardar datos**: partidas validadas y eventos de analítica.
3. **Entregar estadísticas** (las que muestra el botón DEV).

¿Por qué no basta con el navegador?

El brief del cliente lo pide explícitamente: "*Validación en servidor de puntuaciones y progreso: el estado del cliente no debe ser de confianza.*" Cualquiera puede abrir la consola del navegador (F12) y cambiar sus puntos. Por eso la **última palabra** la tiene el servidor.

Una analogía: el árbitro

> **El jugador (navegador)** juega su partida. Al terminar le entrega al árbitro una hoja con la **semilla** de los dados y la **lista de jugadas**.
>
> No le dice "gané 5 puntos".

> **El árbitro (la API)** repite la partida él solo, con los **mismos dados** (misma semilla) y las **mismas reglas**. Si todas las jugadas son legales, anota el resultado que **él** calculó en el libro de registros (la base de datos).

¿Por qué el árbitro obtiene los mismos dados?

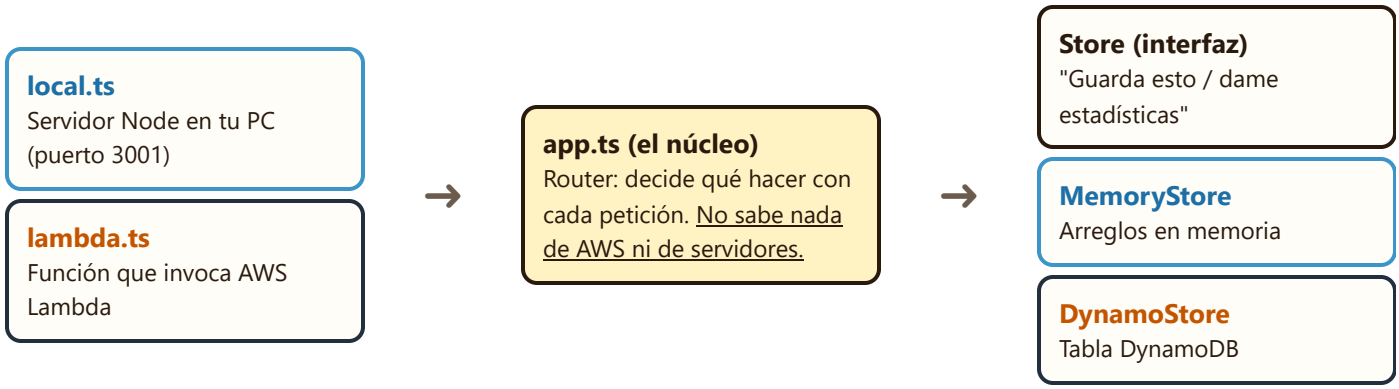
Los dados no usan `Math.random()`, sino un generador con **semilla** (Mulberry32, en `packages/engine/src/rng.ts`). Con la misma semilla, la secuencia de números es **idéntica** en cualquier computadora. Por eso el servidor puede reconstruir la partida exacta.

¿Por qué usa las mismas reglas?

Porque la API importa **el mismo paquete** `@snac-hunters/engine` que la web. No hay dos copias de las reglas que puedan desincronizarse.

2. La idea de diseño: un núcleo y dos adaptadores

La API necesita correr en **dos lugares**: en tu computadora mientras desarrollas y en **AWS Lambda** en producción. En lugar de escribir dos APIs, se separa así:



- **Adaptador** (`local.ts` / `lambda.ts`): traduce la petición del entorno al formato común `ApiRequest` y traduce la respuesta de vuelta.
- **Núcleo** (`app.ts`): toda la lógica: validar, re-simular y decidir la respuesta.
- **Almacenamiento** (`Store`): una interfaz con dos implementaciones. El núcleo recibe "algún `Store`" y no le importa cuál es.

Beneficios:

- Se puede desarrollar y **probar sin una cuenta de AWS**.
- Las pruebas de integración ejercitan el núcleo real, no una copia.
- Pasar a Lambda **no requiere cambiar ni una línea de** `app.ts` (Parte B).
- Si mañana se cambia DynamoDB por otra base de datos, solo se escribe otro `Store`.

Este patrón se conoce como "puertos y adaptadores" o *arquitectura hexagonal*.

Los archivos

Archivo	Dónde corre	Responsabilidad
`src/game.ts`	ambos	Carga y valida `config/` (monstruos, dados, reglas).
`src/app.ts`	ambos	**Núcleo**: router, validación con Zod, re-simulación y cálculo de puntos.
`src/store.ts`	ambos	Interfaz `Store`, tipos de registros, `summarize()` y `MemoryStore`.
`src/dynamo-store.ts`	AWS	`DynamoStore`: implementación de `Store` sobre DynamoDB.
`src/lambda.ts`	AWS	Adaptador: evento de API Gateway → `ApiRequest` → respuesta con CORS.
`src/local.ts`	local	Adaptador: servidor HTTP de Node en el puerto 3001 con `MemoryStore`.

`src/__tests__/app.test.ts`	tests	9 pruebas de integración del núcleo.

3. Recorrido completo de una partida validada

Esto es lo que pasa cuando terminas un combate y aparece "*✓ Validado por el servidor*":

1. **Navegador.** `apps/web/src/game/store.ts` llama a `submitRun()` (`apps/web/src/game/api.ts`), que hace `POST /runs` con **solo** la semilla y la lista de acciones.
2. **Adaptador.** `local.ts` (o `lambda.ts` en AWS) arma un `ApiRequest` `{ method, path, body, userId }` y llama a `handleRequest()`.
3. **Router.** `handleRequest` forma la clave `"POST /runs"` y llama a `submitRun()`.
4. **Validación de formato.** Zod revisa que el body tenga la forma correcta. Si no → `400`.
5. **Búsqueda del monstruo** en los datos de `config/monsters.json`. Si no existe → `404`.
6. **Re-simulación.** `replayCombat(monstruo, serverSetup, seed, actions, diceTable)` repite la partida con el engine. Si una acción es ilegal, el engine lanza `CombatError` → `422`.
7. **¿Terminó el combate?** Si el estado final no está resuelto → `422`.
8. **Cálculo del servidor:** resultado, KO, trofeo y puntos (con `scoring` de `config/game.json`).
9. **Guardado.** `store.saveRun(run)` lo escribe en memoria (local) o en DynamoDB (AWS).
10. **Respuesta** `201` con el registro. La web muestra el aviso verde.

Ejemplo real de petición

```
POST /runs
Content-Type: application/json

{
  "monsterId": "garden-salaad",
  "seed": "snac-hunters-tutorial",
  "actions": [
    { "type": "assign", "dieId": "d4", "slot": 0 },
    { "type": "assign", "dieId": "d0", "slot": 2 },
    { "type": "reroll", "dieIds": ["d2", "d3"] },
    { "type": "assign", "dieId": "d5", "slot": 1 },
    { "type": "assign", "dieId": "d1", "slot": 3 },
    { "type": "resolve", "trophy": "Claw" }
  ]
}
```

Respuesta si todo es legal (201)

```
{
  "valid": true,
  "runId": "3f0c…",              ← UUID generado por el servidor
  "userId": "guest",              ← en AWS: el "sub" del usuario de Cognito
  "monsterId": "garden-salaad",
  "seed": "snac-hunters-tutorial",
  "outcome": "defeat",            ← lo calculó el SERVIDOR
  "ko": false,
  "trophy": "Claw",
  "points": 1,                    ← de config/game.json → scoring
  "rerolls": 1,
  "createdAt": "2026-10-14T18:03:11.000Z",
  "hp": 6
}
```

Respuesta si el cliente hace trampa (422)

```
{ "valid": false, "error": "El dado \"d0\" no está en la reserva" }
```

Con la semilla del tutorial, `d0` sale hacha: intentar ponerlo en dos espacios produce este error. Poner una cara en un espacio que no le corresponde produce otro, por ejemplo `La cara "heart" no cubre el slot "all"`.

4. Archivo por archivo

4.1 `game.ts`: carga la configuración

```
import rawDice from '@snac-hunters/config/dice.json';
import rawGame from '@snac-hunters/config/game.json';
import rawMonsters from '@snac-hunters/config/monsters.json';

export const diceTable = loadDiceTable(rawDice);       // valida con Zod
export const gameConfig = loadGameConfig(rawGame);
export const monsters = loadMonsters(rawMonsters);

/** El setup lo decide el servidor, nunca el cliente. */
export const serverSetup = {
  dice: gameConfig.player.startingDice,   // 4 rojos + 1 verde
  hp: gameConfig.player.startingHp,       // 6
  maxHp: gameConfig.player.maxHp,
};
```

- Se ejecuta **una sola vez** al arrancar (en Lambda, una vez por contenedor; las siguientes peticiones lo reutilizan).
- Si algún JSON es inválido, la API **no arranca**. Es preferible fallar rápido a jugar con datos corruptos.
- `serverSetup` es clave para la seguridad: el cliente **no puede** decir "empecé con 20 dados y 99 de vida".
- Los JSON se importan directamente; al empaquetar, esbuild los incrusta en el código de la Lambda.

4.2 `app.ts`: el núcleo

Los contratos de entrada y salida

```
export interface ApiRequest {
  method: string;          // "GET", "POST"…
  path: string;            // "/runs"
  body: string | null;     // JSON como texto
  userId: string;          // Cognito "sub" en AWS; "guest" en local
}
export interface ApiResponse { statusCode: number; body: unknown; }
```

Son tipos **propios**, no los de AWS. Por eso `app.ts` no depende de Lambda.

El router

```
export async function handleRequest(req, store) {
  const route = `${req.method.toUpperCase()} ${req.path.replace(/\/+$/, '') || '/'}`;
  switch (route) {
    case 'GET /health':  return { statusCode: 200, body: { ok: true, monsters: monsters.length } };
    case 'POST /runs':   return submitRun(req, store);
    case 'POST /events': ... // valida y guarda eventos → 202
    case 'GET /stats':   return { statusCode: 200, body: await store.stats() };
    default:            return { statusCode: 404, ... };
  }
}
```

- Combina método y ruta en una sola clave (`"POST /runs"`), lo que hace el `switch` fácil de leer.
- El `.replace(/\/+$/, '')` quita las "/" finales: `/health/` y `/health` funcionan igual.
- `store` llega como **parámetro** (inyección de dependencias). El núcleo no crea su propia base de datos.

`submitRun`: el corazón de la seguridad

```
const parsed = RunSubmissionSchema.safeParse(parseJson(req.body));
if (!parsed.success) return { statusCode: 400, ... };          // 1. formato

const monster = monsters.find((m) => m.id === monsterId);
if (!monster) return { statusCode: 404, ... };                  // 2. monstruo

try {
  const final = replayCombat(monster, serverSetup, seed, actions, diceTable); // 3. re-simular
  if (final.status !== 'resolved') return { statusCode: 422, ... };      // 4. ¿terminó?
  const koPenalty = final.ko && final.outcome !== 'ko' ? gameConfig.scoring.ko : 0;
  const run = { runId: randomUUID(), userId: req.userId, ...,
                points: gameConfig.scoring[final.outcome] + koPenalty, ... };      // 5. calcular
  await store.saveRun(run);                                                  // 6. guardar
  return { statusCode: 201, body: { valid: true, ...run, hp: final.hp } };
} catch (e) {
  if (e instanceof CombatError) return { statusCode: 422, ... };   // acción ilegal
  throw e;                                                        // error inesperado → 500
}
```

- **Puntos:** derrota +1, estudiar 0, huir 0, KO −1 (según `scoring`). Si te derrotan y derrotas al monstruo en la misma tirada, se suma la derrota **y** se resta el KO.
- Un `CombatError` es **culpa del cliente** (422). Cualquier otro error es **culpa del servidor** y se relanza para que el adaptador responda 500.
- `parseJson` devuelve `undefined` si el JSON está roto; así Zod lo rechaza con 400 en lugar de tronar.

4.3 `store.ts`: LA INTERFAZ DE ALMACENAMIENTO

```ts
export interface Store {
  saveRun(run: RunRecord): Promise<void>;
  saveEvents(events: readonly EventRecord[]): Promise<void>;
  stats(): Promise<Stats>;
}
```

Elemento	Qué es
`RunRecord`	Una partida validada: runId, userId, monsterId, seed, outcome, ko, trophy, points, rerolls, createdAt.
`EventRecord`	Un evento de analítica: sessionId, userId, type (`session_start`, `dice_rolled`…), at (timestamp) y data.
`Stats`	El resumen: total de partidas, puntos, resultados, sesiones, eventos por tipo y promedio de relanzamientos.
`summarize()`	Función pura que calcula `Stats` a partir de listas de partidas y eventos. La comparten las dos implementaciones.
`MemoryStore`	Guarda en arreglos. Rápido y sin configuración, pero **se borra al reiniciar** el servidor local.

Todos los métodos son `async` (devuelven `Promise`) aunque `MemoryStore` no lo necesite. Así ambas implementaciones son intercambiables, porque DynamoDB sí es asíncrono.

4.4 `dynamo-store.ts`: DYNAMODB

Implementa la misma interfaz `Store` con el **AWS SDK v3**. Usa `DynamoDBDocumentClient`, que convierte objetos de JavaScript al formato de DynamoDB automáticamente.

Diseño single-table (una sola tabla para todo)

Tipo	pk (partition key)	sk (sort key)	kind
Partida	`RUN#<userId>`	`<createdAt>#<runId>`	`run`
Evento	`EVENT#<sessionId>`	`<at>#<n>`	`event`

- La **pk** agrupa: todas las partidas de un usuario quedan juntas, igual que todos los eventos de una sesión.
- La **sk** empieza con la fecha, así que los registros quedan **ordenados cronológicamente** dentro de cada grupo.
- `kind` permite distinguir qué es cada registro al leer la tabla completa.

Los tres métodos

Método	Comando de DynamoDB	Detalle
`saveRun`	`PutCommand`	Una escritura por partida.

`saveEvents`	`BatchWriteCommand`	Escribe en bloques de **25**, que es el máximo que permite DynamoDB por llamada. Un lote de 100 eventos son 4 llamadas.
`stats`	`ScanCommand`	Lee **toda** la tabla página por página (DynamoDB devuelve hasta 1 MB por página; `LastEvaluatedKey` indica que hay más).

> **Limitación conocida:** `Scan` lee la tabla completa en cada llamada. Con pocos datos (demo) está bien; con muchos jugadores sería lento y costoso. El plan documentado es enviar los eventos a **Firehose** → **S3** → **Athena** para la analítica a gran escala.

El nombre de la tabla llega en el constructor; `lambda.ts` lo toma de la variable de entorno `TABLE_NAME`, que define CDK.

4.5 `lambda.ts`: el adaptador para AWS

```
const store = new DynamoStore(process.env.TABLE_NAME ?? '');   // se crea 1 vez por contenedor

export async function handler(event: APIGatewayProxyEvent) {
  const claims = event.requestContext.authorizer?.claims;    // datos del usuario (Cognito)
  try {
    const res = await handleRequest({
      method: event.httpMethod, path: event.path, body: event.body,
      userId: claims?.sub ?? 'guest',
    }, store);
    return { statusCode: res.statusCode, headers: CORS, body: JSON.stringify(res.body) };
  } catch (e) {
    console.error(e);                                          // → CloudWatch Logs
    return { statusCode: 500, headers: CORS, body: '{"error":"Error interno"}' };
  }
}
```

Cómo traduce el evento de API Gateway

Campo del evento de API Gateway	→ campo de `ApiRequest`
`event.httpMethod`	`method`
`event.path` (sin el nombre del stage)	`path`
`event.body`	`body`
`event.requestContext.authorizer.claims.sub`	`userId` (id único del usuario en Cognito)

- Confía en `claims` porque el **authorizer de API Gateway ya verificó la firma del token** antes de invocar la Lambda. Si el token es inválido, la Lambda ni siquiera se ejecuta.
- Agrega los encabezados **CORS** a cada respuesta para que el navegador acepte respuestas de otro dominio.
- Ante un error inesperado responde 500 **sin** revelar detalles internos; el detalle queda en CloudWatch.

4.6 `local.ts`: el adaptador para tu PC

- Crea un servidor con `node:http` en el puerto **3001** (o el de la variable `PORT`).
- Responde a `OPTIONS` con 204 (preflight de CORS que hace el navegador).
- Junta el body, llama a `handleRequest` con `userId: 'guest'` y un `MemoryStore`, e imprime cada petición en la terminal: `POST /runs -> 201`.
- Se ejecuta con `pnpm --filter @snac-hunters/api dev` → `tsx watch src/local.ts`. **tsx** corre TypeScript sin compilar y **watch** reinicia al guardar cambios.

5. Referencia de endpoints

Endpoint	Auth en AWS	Body	Respuestas
`GET /health`	Pública	—	`200 { ok: true, monsters: 9 }`
`POST /runs`	Cognito	`{ monsterId, seed, actions[] }`	`201` válida · `400` formato · `404` monstruo · `422` ilegal o sin terminar
`POST /events`	Cognito	`{ sessionId, events: [{ type, at, data? }] }` (1 a 100)	`202 { accepted: n }` · `400`
`GET /stats`	Cognito	—	`200 { runs, points, outcomes, sessions, events, avgRerolls }`
otra ruta	—	—	`404` (en AWS, API Gateway responde `403 Missing Authentication Token` antes de llegar a la Lambda)

En local **no hay autenticación**: todo entra como `guest`.

Las acciones que acepta `actions[]`

Acción	Significado
`{ "type": "assign", "dieId": "d4", "slot": 0 }`	Coloca el dado `d4` en el espacio 0 (de 0 a 3).
`{ "type": "unassign", "slot": 0 }`	Quita el dado del espacio 0.
`{ "type": "reroll", "dieIds": ["d2","d3"] }`	Relanza esos dados (y agrega un dado blanco).
`{ "type": "resolve", "trophy": "Claw" }`	Termina el combate; `trophy` es obligatorio al derrotar o estudiar.

Los dados se identifican por orden de creación: `d0` … `d4` son los iniciales y cada relanzamiento crea el siguiente (`d5`, `d6`…), que es el dado blanco.

6. Validación de entrada y códigos HTTP

Todo lo que llega del cliente pasa por **Zod** antes de usarse. Los límites evitan abusos (por ejemplo, mandar un millón de acciones para saturar la Lambda):

Campo	Regla
`monsterId`	texto de 1 a 64 caracteres
`seed`	texto de 1 a 128 caracteres
`actions`	máximo 300; cada una debe ser uno de los 4 tipos exactos

`dieId`	máximo 8 caracteres
`reroll.dieIds`	de 1 a 40 dados
`trophy`	solo `Horn`, `Wing`, `Claw` o `Tail`
`events`	de 1 a 100 por lote; `type` de hasta 40 caracteres; `at` entero ≥ 0

Los campos extra se **descartan**. Si el cliente manda `"points": 999`, Zod lo elimina.

Código	Significado	Cuándo
200	OK	`/health`, `/stats`
201	Creado	Partida válida y guardada
202	Aceptado	Eventos de analítica recibidos
400	Petición mal formada	JSON roto o que no cumple el esquema
401	No autenticado	Solo en AWS: falta el token de Cognito o no es válido
404	No encontrado	Monstruo inexistente o ruta desconocida
422	Entidad no procesable	Formato correcto pero **jugada ilegal** o combate sin terminar
500	Error interno	Fallo inesperado del servidor (revisar logs)

7. Pruebas

```
pnpm --filter @snac-hunters/api test        # → 9 passed
```

Las pruebas (`src/__tests__/app.test.ts`) llaman a `handleRequest` directamente con un `MemoryStore` nuevo en cada prueba. Son **pruebas de integración**: ejercitan juntos el router, Zod, el engine real y el almacenamiento.

Prueba	Qué demuestra
config/ real es válida	Los 9 monstruos, la tabla de dados, `game.json` y `tutorial.json` pasan los esquemas.
Re-simula y guarda	Un "cliente honesto" simulado juega y la API acepta la partida con el mismo resultado.
Combate sin terminar	Responde 422 y no guarda nada.
Cliente tramposo	Usar el mismo dado dos veces → 422; nada se guarda.
Ignora resultados impuestos	Mandar `outcome` y `points: 999` no tiene efecto.
Payload inválido	JSON roto → 400; monstruo inexistente → 404; acción desconocida → 400.
Analítica	`/events` guarda y `/stats` resume sesiones y tipos de evento.
Lote vacío	`/events` sin eventos → 400.
Rutas	`/health/` con "/" final funciona; método no soportado → 404.

> **¿Por qué no hay pruebas de `DynamoStore`?** Requieren DynamoDB real o un emulador (DynamoDB Local / LocalStack). Es un pendiente razonable; el núcleo, que es lo crítico, está cubierto.

8. Probar la API a mano

Con la API local corriendo (`pnpm --filter @snac-hunters/api dev`):

Desde el navegador (solo GET)

- `http://localhost:3001/health` → `{"ok":true,"monsters":9}`
- `http://localhost:3001/stats` → resumen (se llena conforme juegas)

Desde Git Bash (curl)

```
# Enviar un evento de analítica
curl -s -X POST http://localhost:3001/events \
  -H "Content-Type: application/json" \
  -d '{"sessionId":"prueba","events":[{"type":"session_start","at":1}]}'
# → {"accepted":1}

# Intentar una partida sin jugadas (combate sin terminar)
curl -s -X POST http://localhost:3001/runs \
  -H "Content-Type: application/json" \
  -d '{"monsterId":"garden-salaad","seed":"x","actions":[]}'
# → {"valid":false,"error":"El combate no ha terminado"}

# Hacer trampa: usar el mismo dado (un hacha) en dos espacios
curl -s -X POST http://localhost:3001/runs \
  -H "Content-Type: application/json" \
  -d '{"monsterId":"garden-salaad","seed":"snac-hunters-tutorial","actions":
[{"type":"assign","dieId":"d0","slot":2},{"type":"assign","dieId":"d0","slot":3}]}'
# → {"valid":false,"error":"El dado \"d0\" no está en la reserva"}
```

Desde PowerShell

```
Invoke-RestMethod http://localhost:3001/health
Invoke-RestMethod -Method Post -Uri http://localhost:3001/events -ContentType 'application/json' `
  -Body '{"sessionId":"prueba","events":[{"type":"session_start","at":1}]}'
```

En Windows PowerShell 5.1, `Invoke-RestMethod` lanza una excepción con los códigos 4xx. Para ver errores como el 422 es más cómodo usar curl desde Git Bash.

> **Tip:** la terminal de la API imprime cada petición (`POST /runs -> 201`). Mientras juegas, mírala para ver la validación en tiempo real.

Parte B

Del servidor local a AWS Lambda

9. Qué cambia y qué no

> **La idea central:** para pasar a AWS **no se modifica la lógica**. `app.ts`, `game.ts`, `store.ts` y el engine son exactamente los mismos. Solo cambia **quién llama** al núcleo (el adaptador) y **dónde se guarda** (el `Store`). Esa conexión ya está escrita en `lambda.ts` y en la infraestructura de CDK.

Aspecto	Local	AWS
Punto de entrada	`local.ts` (servidor Node)	`lambda.ts` (función `handler`)
Quién recibe HTTP	Tu PC, puerto 3001	**API Gateway** (URL pública HTTPS)
Almacenamiento	`MemoryStore` (se borra al reiniciar)	`DynamoStore` → tabla **DynamoDB** (permanente)
Usuario	Siempre `guest`	`sub` del token de **Cognito**
Autenticación	Ninguna	**Cognito authorizer** en `/runs`, `/events` y `/stats`
Logs	La terminal	**CloudWatch Logs** (30 días)
Servidor encendido	Mientras la terminal esté abierta	Siempre disponible; AWS crea contenedores por demanda
Costo	Gratis	Pago por uso (sección 15)
URL que usa la web	`http://localhost:3001` (default)	`VITE_API_URL` = salida `ApiUrl` de CDK

Lo único que hay que "cambiar"

1. **Desplegar** la infraestructura (crea API Gateway, Lambda, DynamoDB, Cognito, S3 y CloudFront).
2. **Decirle a la web** la nueva dirección de la API y los datos de Cognito (archivo `.env.production`).
3. **Resolver el login**: en AWS las rutas piden un usuario autenticado (sección 13).

10. Cómo CDK convierte el código en una Lambda

En `infra/lib/snac-hunters-stack.ts`:

```
const apiFn = new NodejsFunction(this, 'ApiFunction', {
  entry: path.join(ROOT, 'apps/api/src/lambda.ts'),   // ← el adaptador de AWS
  handler: 'handler',                                 // ← la función exportada
  runtime: lambda.Runtime.NODEJS_22_X,
  memorySize: 256, timeout: Duration.seconds(10),
  environment: { TABLE_NAME: table.tableName, STAGE: props.stage },
  bundling: {
    format: OutputFormat.ESM, target: 'node22',
    externalModules: ['@aws-sdk/*'],                  // ya viene incluido en Lambda
    banner: "import{createRequire}from'module';const require=createRequire(import.meta.url);",
  },
});
table.grantReadWriteData(apiFn);                      // permiso IAM: solo esta tabla
```

Pieza	Qué hace
`NodejsFunction`	Al hacer `synth` / `deploy`, ejecuta **esbuild** sobre `lambda.ts` y empaqueta en **un solo archivo** todo lo que importa: `app.ts`, el engine, Zod y los JSON de `config/`.
`externalModules: ['@aws-sdk/*']`	No incluye el SDK de AWS en el paquete porque el runtime de Lambda ya lo trae. El paquete resultante pesa unos 140 KB.
`format: ESM` + `banner`	El proyecto usa módulos ES. El *banner* crea `require` para las librerías que aún lo usan internamente.
`environment.TABLE_NAME`	CDK inyecta el nombre real de la tabla (generado por AWS). `lambda.ts` lo lee con `process.env.TABLE_NAME`.
`grantReadWriteData`	Crea el rol IAM con permisos **solo** sobre esa tabla (principio de mínimo privilegio).
`LambdaIntegration`	Conecta cada ruta de API Gateway (`/health`, `/runs`…) con la Lambda (modo *proxy*: le pasa la petición completa).
`CognitoUserPoolsAuthorizer`	Valida el token antes de llamar a la Lambda en `/runs`, `/events` y `/stats`.
`addGatewayResponse` (4XX/5XX)	Agrega CORS a los errores que genera API Gateway (como el 401), para que el navegador muestre el error real.

Una sola Lambda atiende todas las rutas: el router de `app.ts` decide. Esto se conoce como "Lambda monolítica" o *lambdalith*; es simple y suficiente para esta escala.

11. Requisitos previos

Requisito	Cómo
Cuenta de AWS	Crear en aws.amazon.com (pide tarjeta). Revisen si su institución o AWS Academy les da créditos.
Usuario IAM (no root)	En la consola: IAM → Users → crear usuario con permisos para desplegar (para el proyecto escolar, `AdministratorAccess`). Crear un **Access key** de tipo CLI. **Nunca** usen las llaves de la cuenta root ni suban llaves a GitHub.
AWS CLI v2	Instalar desde la documentación oficial de AWS (instalador MSI para Windows). Verificar con `aws --version`.
Configurar credenciales	`aws configure` → pegar Access Key, Secret, región `us-east-1` y formato `json`.
Alerta de presupuesto (recomendado)	Billing → Budgets → crear un presupuesto de, por ejemplo, 5 USD con alerta por email.
Lo del proyecto	`pnpm install` ya hecho. **No** se necesita Docker: CDK usa el esbuild instalado en el proyecto.

```
# Verificar que las credenciales funcionan (muestra tu número de cuenta)
aws sts get-caller-identity
```

12. Despliegue paso a paso

> **Ojo con el comando:** se escribe `pnpm --filter @snac-hunters/infra run deploy`. Sin `run`, pnpm ejecuta su propio comando integrado `pnpm deploy`, que hace otra cosa.

Paso 1: Bootstrap (una sola vez por cuenta y región)

```
pnpm --filter @snac-hunters/infra exec cdk bootstrap
```

Crea en tu cuenta los recursos que CDK necesita para subir archivos (un bucket S3 y roles IAM, en un stack llamado `CDKToolkit`). Si te saltas este paso, el deploy falla con `SSM parameter /cdk-bootstrap/... not found`.

Paso 2: Revisar qué se va a crear

```
pnpm --filter @snac-hunters/infra synth -c stage=dev    # genera la plantilla, sin tocar AWS
pnpm --filter @snac-hunters/infra exec cdk diff -c stage=dev   # qué cambiaría en tu cuenta
```

Paso 3: Primer despliegue (API, base de datos y login)

```
pnpm --filter @snac-hunters/infra run deploy -c stage=dev --outputs-file cdk-outputs.json
```

CDK muestra los cambios de seguridad (roles IAM) y pregunta `Do you wish to deploy these changes (y/n)?` → **y**. Tarda varios minutos (CloudFront es lo más lento). Al final imprime:

```
Outputs:
SnacHunters-dev.ApiUrl           = https://abc123.execute-api.us-east-1.amazonaws.com/dev/
SnacHunters-dev.UserPoolId       = us-east-1_XXXXXXXXX
SnacHunters-dev.UserPoolClientId = 1a2b3c4d5e6f...
SnacHunters-dev.WebUrl           = https://d1234abcd.cloudfront.net
```

Los mismos valores quedan guardados en `infra/cdk-outputs.json`.

Paso 4: Probar la Lambda

```
curl https://abc123.execute-api.us-east-1.amazonaws.com/dev/health
# → {"ok":true,"monsters":9}   ← ¡tu código corriendo en Lambda!

curl -X POST https://abc123.execute-api.us-east-1.amazonaws.com/dev/runs
# → {"message":"Unauthorized"}  ← correcto: el authorizer de Cognito bloquea sin token
```

Paso 5: Apuntar la web a AWS

Crear `apps/web/.env.production` (basado en `.env.example`; está en `.gitignore` y no se sube a GitHub):

```
VITE_API_URL=https://abc123.execute-api.us-east-1.amazonaws.com/dev
VITE_USER_POOL_ID=us-east-1_XXXXXXXXX
VITE_USER_POOL_CLIENT_ID=1a2b3c4d5e6f...
```

Vite inserta estas variables **al compilar**. Si las cambias, hay que volver a hacer build. La "/" final de la URL se quita automáticamente en `config.ts`.

Paso 6: Compilar la web y desplegar de nuevo

```
pnpm --filter @snac-hunters/web build                      # genera apps/web/dist
pnpm --filter @snac-hunters/infra run deploy -c stage=dev   # sube dist/ a S3 e invalida CloudFront
```

El stack solo sube la web si existe `apps/web/dist` (por eso hay dos despliegues: el primero crea la API cuyos datos necesita la web).

Paso 7: Abrir el juego

Abrir el `WebUrl` (`https://d1234abcd.cloudfront.net`) en la computadora o en el celular, desde cualquier red.

Despliegues siguientes

Después de cambiar código: `web build` (si cambió la web) y luego `run deploy`. CDK solo actualiza lo que cambió.

13. El pendiente del login (importante)

> **Situación actual:** en AWS, `/runs`, `/events` y `/stats` exigen un usuario autenticado con Cognito. La web ya sabe **enviar** el token (`apps/web/src/game/auth.ts` y `api.ts`), pero **no tiene pantalla de login**. Si se despliega tal cual, el juego funciona, pero al terminar un combate el servidor responderá **401** y la tarjeta mostrará "*✗ El servidor rechazó la partida*".

Opción A (recomendada): agregar login con Amplify UI

```
pnpm --filter @snac-hunters/web add @aws-amplify/ui-react
```

```
// apps/web/src/main.tsx
import { Authenticator } from '@aws-amplify/ui-react';
import '@aws-amplify/ui-react/styles.css';
import { authEnabled } from './game/auth';   // también configura Amplify

createRoot(document.getElementById('root')!).render(
  <StrictMode>
    {authEnabled ? (
      <Authenticator>{() => <App />}</Authenticator>   // pide login antes de jugar
    ) : (
      <App />                                          // local: modo invitado
    )}
  </StrictMode>,
);
```

- `<Authenticator>` trae pantallas listas de registro, verificación por email e inicio de sesión, conectadas al User Pool que creó CDK (que ya permite el autorregistro).
- Una vez iniciada la sesión, `getIdToken()` devuelve el token, `api.ts` lo manda en el encabezado `Authorization` y el authorizer lo acepta.
- En la Lambda, `userId` pasa de `guest` al `sub` real del jugador, así que las partidas quedan asociadas a su cuenta en DynamoDB.
- Se puede crear un usuario de prueba sin la UI: `aws cognito-idp admin-create-user --user-pool-id <UserPoolId> --username correo@ejemplo.com`.

Opción B (temporal, solo para una demo): rutas públicas

```
// infra/lib/snac-hunters-stack.ts: quitar "authed" de runs y events
api.root.addResource('runs').addMethod('POST', integration);     // antes: ..., authed)
api.root.addResource('events').addMethod('POST', integration);   // antes: ..., authed)
api.root.addResource('stats').addMethod('GET', integration, authed);  // stats sigue protegido
```

- La validación anti-trampa **sigue funcionando** (la re-simulación no depende del login), pero todas las partidas se guardan como `guest`.
- **Riesgo:** cualquiera puede llamar a esas rutas. El *throttling* (20 peticiones/s) limita abusos, pero no los evita. **No dejarlo así** en el producto final.

Recomendación: usar B solo si hay que mostrar algo desplegado con urgencia, y planear A para el sprint siguiente.

14. Verificar, depurar y errores comunes

Dónde mirar en la consola de AWS (región us-east-1)

Servicio	Qué revisar
CloudFormation	Stack `SnacHunters-dev`: estado del despliegue y, si falló, el recurso y el motivo en la pestaña *Events*.
CloudWatch → Log groups	Buscar el grupo que contiene `ApiLogs`. Ahí aparecen los `console.error` de la Lambda y cada invocación.
Lambda	Función `…ApiFunction…`: pestaña *Test* para invocarla a mano y *Monitor* para ver errores y duración.
DynamoDB → Explore items	Ver las partidas (`RUN#…`) y los eventos (`EVENT#…`) guardados.
API Gateway	API `snac-hunters-dev`: rutas, authorizer y stage `dev`.
Cognito	User Pool: usuarios registrados.

Errores comunes

Síntoma	Causa probable y solución
`SSM parameter /cdk-bootstrap/... not found`	Falta el bootstrap → paso 1.
`Unable to resolve AWS account` / `ExpiredToken`	Credenciales no configuradas o vencidas → `aws configure` y `aws sts get-caller-identity`.
`AccessDenied` durante el deploy	El usuario IAM no tiene permisos suficientes.
Error `ERR_PNPM_…` o mensajes de "deploy" de pnpm	Faltó `run`: `pnpm --filter @snac-hunters/infra run deploy`.
`{"message":"Unauthorized"}` (401)	Falta el login o el token es inválido → sección 13.
`{"message":"Missing Authentication Token"}` (403)	Ruta o método que no existe en API Gateway (p. ej. falta `/dev` en la URL o hay una errata).
"Error de CORS" en la consola del navegador	Casi siempre esconde un 401/403; revisar la pestaña *Network*. Las *gateway responses* con CORS ya están configuradas para mostrar el error real.
`500 {"error":"Error interno"}`	Excepción en la Lambda → revisar CloudWatch Logs.
`502 Bad Gateway`	La Lambda tronó antes de responder (p. ej. un JSON de `config/` inválido al arrancar) → CloudWatch Logs.
La web sigue llamando a `localhost:3001`	Falta `.env.production` o no se volvió a hacer build después de crearlo.
Cambié la web y no se ve en CloudFront	Faltó `web build` antes del deploy, o es caché del navegador (Ctrl + F5).

"Cold start" (arranque en frío)

La primera petición después de un rato sin uso tarda un poco más (cientos de milisegundos), porque AWS crea un contenedor nuevo y ejecuta `game.ts`. Las siguientes reutilizan el contenedor y son rápidas. Es normal en Lambda.

15. Costos y cómo borrar todo

¿Cuánto cuesta?

Todos los servicios usados son de **pago por uso** y no tienen servidores encendidos esperando: Lambda cobra por invocación y tiempo, DynamoDB está en modo *on-demand*, y API Gateway, CloudFront y S3 cobran por peticiones y transferencia. Con el tráfico de una demo escolar el costo es de **centavos**, y suele quedar dentro de la capa gratuita o de los créditos de la cuenta.

> Las condiciones de la capa gratuita de AWS dependen del tipo y la antigüedad de la cuenta. Revisen **Billing** → **Free Tier** en su cuenta y configuren la **alerta de presupuesto** (sección 11). Así nunca hay sorpresas.

Borrar todo

```
pnpm --filter @snac-hunters/infra exec cdk destroy -c stage=dev
```

Stage	Qué pasa con los datos al destruir
`dev`	Se **borran** la tabla, el User Pool, los logs y el bucket con su contenido (`RemovalPolicy.DESTROY`).
`prod`	Se **conservan** la tabla, el User Pool y el bucket (`RemovalPolicy.RETAIN`); hay que borrarlos a mano si se desea. Además, la tabla de prod tiene recuperación a un punto en el tiempo (PITR).

El stack `CDKToolkit` del bootstrap no se borra con `destroy`; casi no genera costo y se puede eliminar desde CloudFormation si ya no se usará CDK.

16. Resumen para explicarlo

¿Qué es `apps/api`?
"Es el backend. Valida las partidas, guarda los datos y entrega estadísticas."

¿Cómo evita trampas?
"El navegador solo manda la semilla y las jugadas. El servidor repite la partida con el mismo motor de reglas y los mismos dados, gracias a la semilla. Si todo es legal, calcula él mismo el resultado y lo guarda; si no, responde 422."

¿Cómo está organizado?
"Un núcleo, `app.ts`, que no sabe nada de AWS, y dos adaptadores: uno local para desarrollar y uno para Lambda. El almacenamiento es una interfaz con dos versiones: memoria y DynamoDB."

¿Cómo se pasa a AWS?
"Sin tocar la lógica. CDK empaqueta `lambda.ts` con esbuild, crea API Gateway con un authorizer de Cognito, la tabla de DynamoDB con permisos mínimos y el hosting en S3 y CloudFront. Se despliega con `cdk deploy` y la web se apunta a la nueva URL con variables de entorno."

¿Qué falta?
"Desplegarlo con una cuenta de AWS, agregar la pantalla de login para que las rutas protegidas acepten al jugador, y mover la analítica pesada a Firehose, S3 y Athena."

Comandos de bolsillo

Comando	Para qué
`pnpm --filter @snac-hunters/api dev`	API local en el puerto 3001
`pnpm --filter @snac-hunters/api test`	9 pruebas de integración
`aws sts get-caller-identity`	Verificar credenciales de AWS
`pnpm --filter @snac-hunters/infra exec cdk bootstrap`	Preparar la cuenta (una vez)
`pnpm --filter @snac-hunters/infra synth -c stage=dev`	Generar la plantilla sin desplegar
`pnpm --filter @snac-hunters/infra run deploy -c stage=dev`	Desplegar o actualizar
`pnpm --filter @snac-hunters/web build`	Compilar la web antes de desplegarla
`pnpm --filter @snac-hunters/infra exec cdk destroy -c stage=dev`	Borrar todo lo de dev